```python
import numpy as np
import pandas as pd
import plotly.express as px
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split

np.random.seed(42)
n_samples = 1000

loan_ids = [f"LN-{1000 + i}" for i in range(n_samples)]
issue_dates = pd.date_range(
    start="2023-01-01", end="2024-12-31", periods=n_samples
)
loan_amounts = np.random.choice(
    [5000, 10000, 15000, 20000, 25000, 35000], size=n_samples
)
received_amounts = loan_amounts * np.random.uniform(0.7, 1.1, size=n_samples)
regions = np.random.choice(
    ["North", "South", "East", "West", "Central"], size=n_samples
)
loan_status = np.random.choice(
    ["Fully Paid", "Current", "Charged Off"],
    size=n_samples,
    p=[0.65, 0.20, 0.15],
)

df = pd.DataFrame(
    {
        "Loan_ID": loan_ids,
        "Issue_Date": issue_dates,
        "Loan_Amount": loan_amounts,
        "Total Received": received amounts,
```

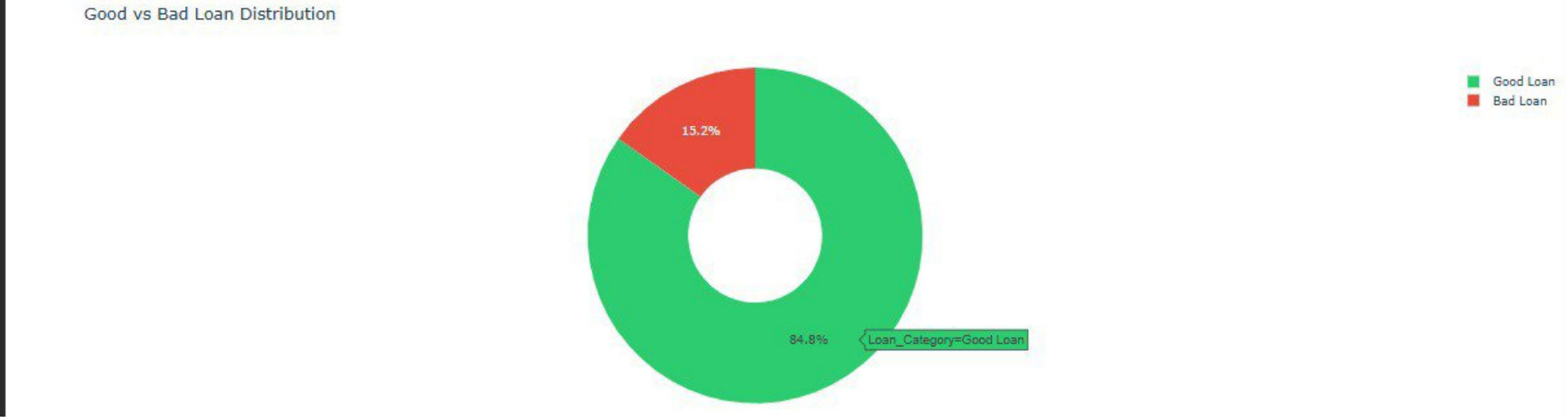
```
        "Region": regions,
        "Loan_Status": loan_status,
    }
)

df["Loan_Category"] = df["Loan_Status"].apply(
    lambda x: "Bad Loan" if x == "Charged Off" else "Good Loan"
)

df["Amount_Category"] = pd.cut(
    df["Loan_Amount"],
    bins=[0, 10000, 20000, 40000],
    labels=["Low", "Medium", "High"],
)

total_applications = len(df)
total_funded_amount = df["Loan_Amount"].sum()
total_received_amount = df["Total_Received"].sum()
good_loans = len(df[df["Loan_Category"] == "Good Loan"])
bad_loans = len(df[df["Loan_Category"] == "Bad Loan"])
bad_loan_pct = (bad_loans / total_applications) * 100

print("=" * 45)
print("          BANK LOAN ANALYSIS SUMMARY          ")
print("=" * 45)
print(f"Total Applications    : {total_applications}")
print(f"Total Funded Amount   : ${total_funded_amount:,.2f}")
print(f"Total Received Amount: ${total_received_amount:,.2f}")
print(f"Good Loans Count      : {good_loans} ({100 - bad_loan_pct:.1f}%)")
print(f"Bad Loans Count       : {bad_loans} ({bad_loan_pct:.1f}%)")
print("=" * 45)
```

```python
fig1 = px.pie(
    df,
    names="Loan_Category",
    title="Good vs Bad Loan Distribution",
    color="Loan_Category",
    color_discrete_map={"Good Loan": "#2ecc71", "Bad Loan": "#e74c3c"},
    hole=0.4,
)
fig1.show()

df["Month_Year"] = df["Issue_Date"].dt.to_period("M").astype(str)
monthly_df = (
    df.groupby("Month_Year")
    .agg({"Loan_Amount": "sum", "Loan_ID": "count"})
    .reset_index()
)

fig2 = px.line(
    monthly_df,
    x="Month_Year",
    y="Loan_Amount",
    title="Monthly Funded Amount Trend",
    markers=True,
    line_shape="spline",
)
fig2.show()

fig3 = px.bar(
    df,
    x="Region",
    y="Loan_Amount",
    color="Loan_Category",
    title="Loan Amount by Region & Category",
```

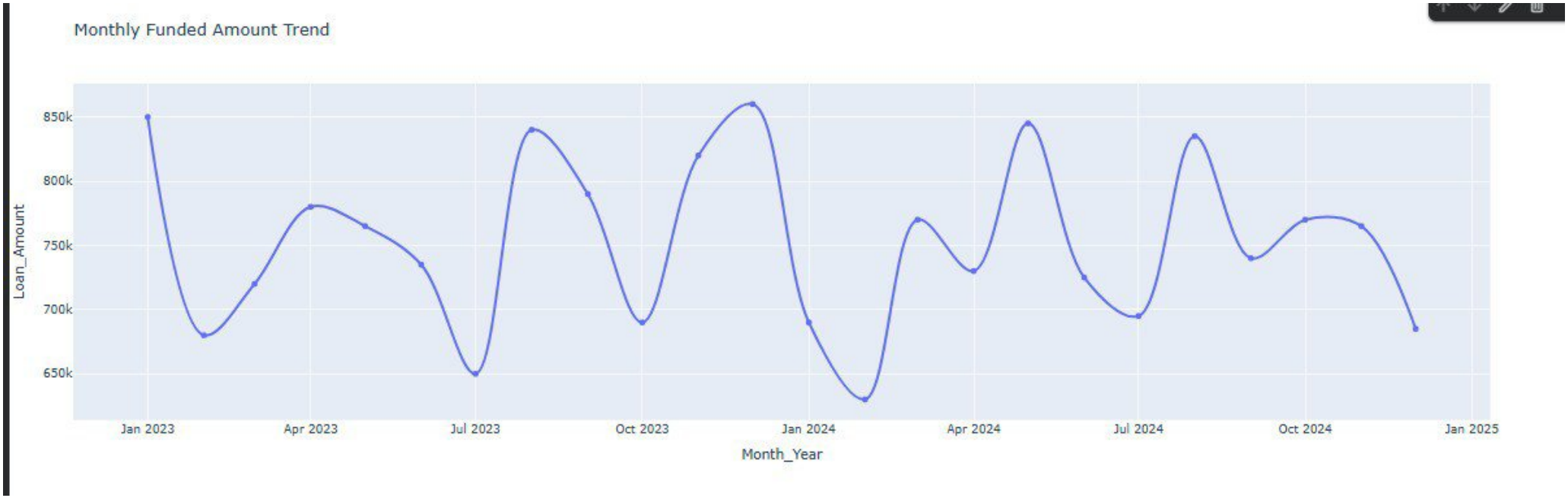
```
    barmode="group",
    color_discrete_map={"Good Loan": "#3498db", "Bad Loan": "#e67e22"},
)
fig3.show()

fig4 = px.treemap(
    df,
    path=["Region", "Amount_Category", "Loan_Category"],
    values="Loan_Amount",
    title="Loan Portfolio Breakdown (Region > Amount Size > Status)",
    color="Loan_Category",
    color_discrete_map={"Good Loan": "#2ecc71", "Bad Loan": "#e74c3c"},
)
fig4.show()

corr = df[["Loan_Amount", "Total_Received"]].corr()
fig5 = px.imshow(
    corr,
    text_auto=True,
    title="Correlation Heatmap",
    color_continuous_scale="Viridis",
)
fig5.show()

fig6 = px.box(
    df,
    x="Loan_Category",
    y="Loan_Amount",
    color="Loan_Category",
    title="Loan Amount Distribution & Outliers by Category",
    points="all",
)
fig6.show()
```

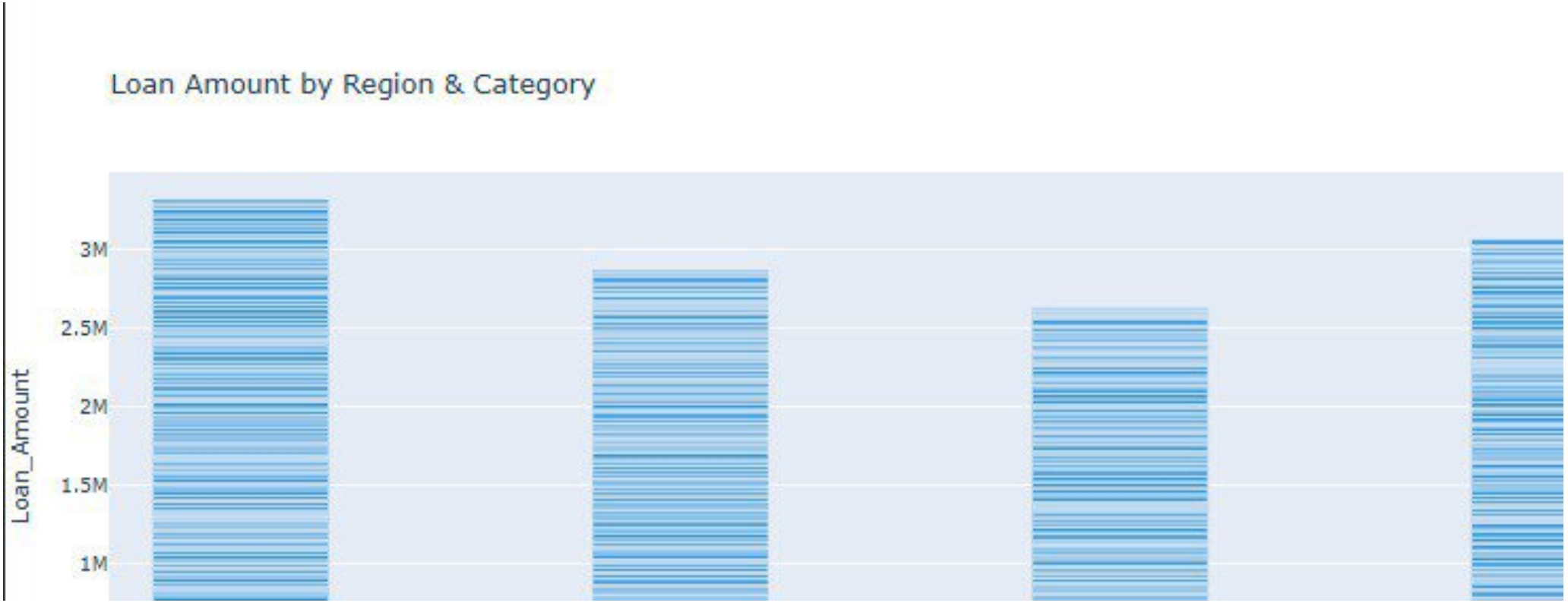
```
df_ml = df.copy()
df_ml["Target"] = (df_ml["Loan_Status"] == "Charged Off").astype(int)
X = pd.get_dummies(
    df_ml[["Loan_Amount", "Total_Received", "Region", "Amount_Category"]],
    drop_first=True,
)
y = df_ml["Target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("\nMachine Learning Model Classification Report:")
print(classification_report(y_test, y_pred))

importances = pd.DataFrame(
    {"Feature": X.columns, "Importance": model.feature_importances_}
).sort_values("Importance", ascending=True)

fig7 = px.bar(
    importances,
    x="Importance",
    y="Feature",
    orientation="h",
    title="Feature Importance for Default Prediction",
)
fig7.show()

df.to_csv("Bank_Loan_Processed_Data.csv", index=False)
print("\nFile 'Bank_Loan_Processed_Data.csv' successfully saved!")
```

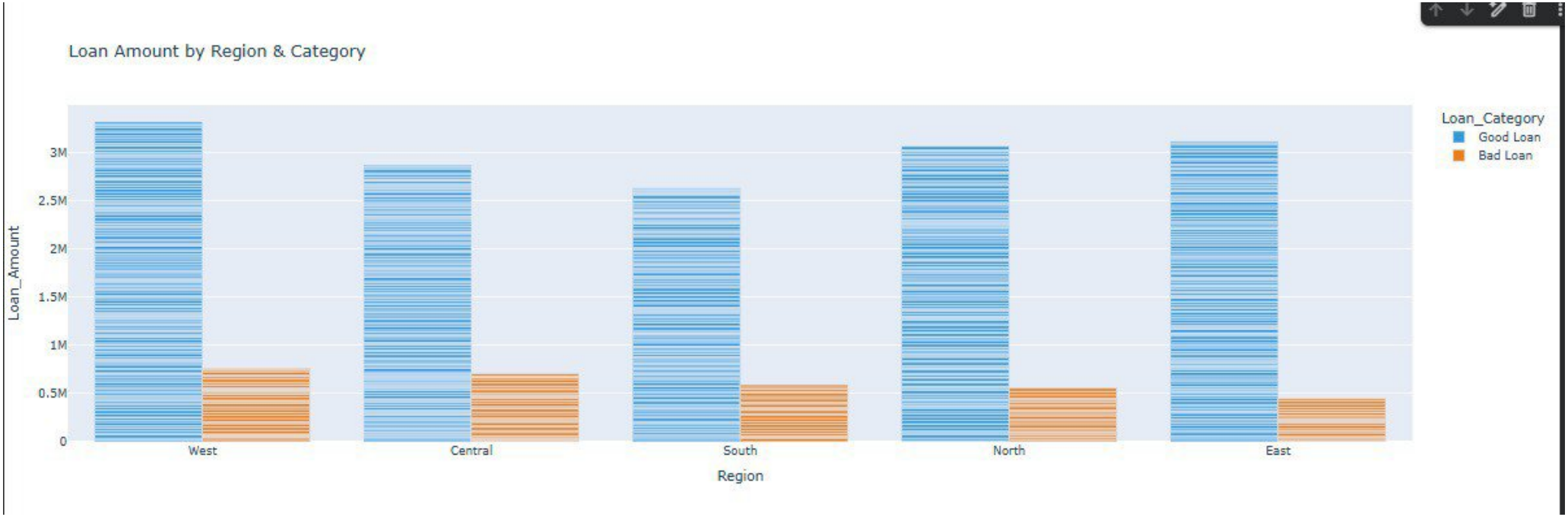
```
==========================================
          BANK LOAN ANALYSIS SUMMARY
==========================================
Total Applications    : 1000
Total Funded Amount   : $18,060,000.00
Total Received Amount: $16,315,411.13
Good Loans Count      : 848 (84.8%)
Bad Loans Count       : 152 (15.2%)
==========================================
```

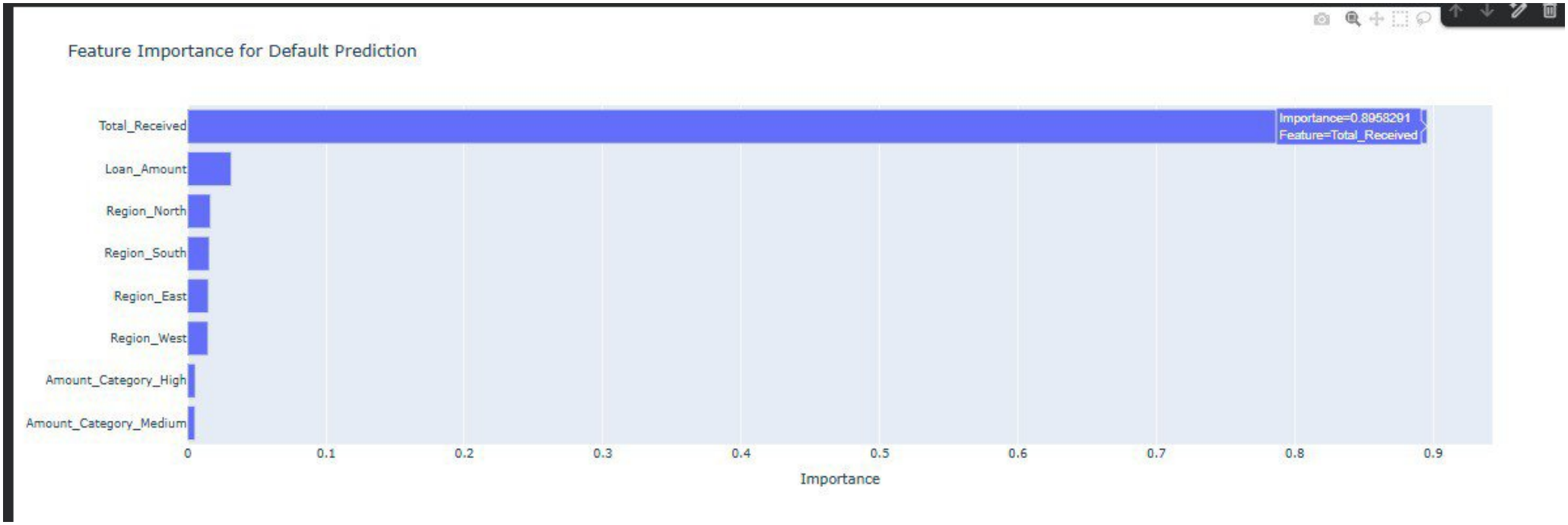


```
sr/local/lib/python3.12/dist-packages/plotly/express/_core.py:1727: FutureWarning:

e default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence thi

sr/local/lib/python3.12/dist-packages/plotly/express/_core.py:1727: FutureWarning:

e default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence thi
```


Monthly Funded Amount Trend
Loan_Amount
850k
800k
750k
700k
650k
Jan 2023
Apr 2023
Jul 2023
Oct 2023
Jan 2024
Apr 2024
Jul 2024
Oct 2024
Jan 2025
Month_Year


Loan Amount by Region & Category
Loan_Amount
3M
2.5M
2M
1.5M
1M

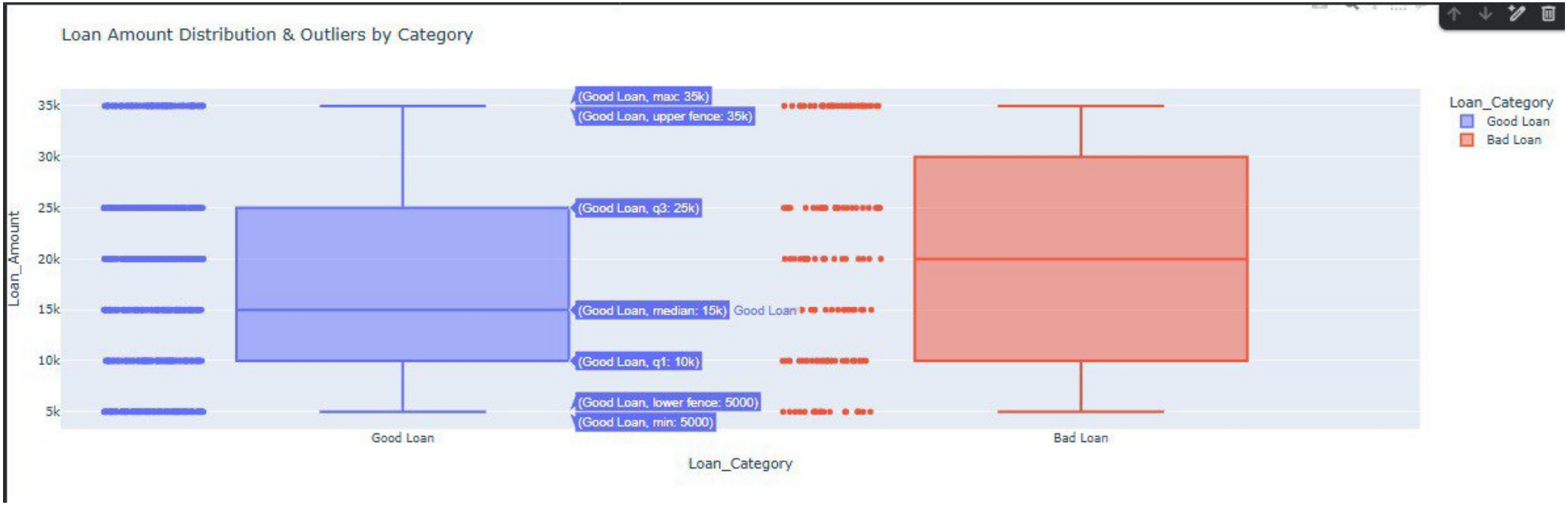

Loan Amount by Region & Category
Loan_Amount
3M
2.5M
2M
1.5M
1M
0.5M
0
West
Central
South
North
East
Region
Loan_Category
Good Loan
Bad Loan


Feature Importance for Default Prediction
Total_Received
Loan_Amount
Region_North
Region_South
Region_East
Region_West
Amount_Category_High
Amount_Category_Medium
Importance=0.8958291
Feature=Total_Received
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
Importance

Correlation Heatmap

Loan Amount Distribution & Outliers by Category

```
nine Learning Model Classification Report:
              precision    recall  f1-score   support

           0       0.86      0.80      0.83       170
           1       0.17      0.23      0.20        30

    accuracy                           0.71       200
   macro avg       0.51      0.52      0.51       200
ghted avg       0.75      0.71      0.73       200
```


Loan Amount Distribution & Outliers by Category
Loan_Amount
35k
30k
25k
20k
15k
10k
5k
(Good Loan, max: 35k)
(Good Loan, upper fence: 35k)
(Good Loan, q3: 25k)
(Good Loan, median: 15k)
Good Loan
(Good Loan, q1: 10k)
(Good Loan, lower fence: 5000)
(Good Loan, min: 5000)
Good Loan
Bad Loan
Loan_Category
Loan_Category
Good Loan
Bad Loan